

Software Test Plan (STP)

Projekt 6: API für den Digitalen Produktpass (DPP) im BaSyx Framework

Customer

Name	Mail
Markus Rentschler	rentschler@lehre.dhbw-stuttgart.de
Pawel Wojcik	pawel.wojcik@lehre.dhbw-stuttgart.de

Dokumenthistorie

Version	Autor	Datum	Kommentar
1.0	Manuel Lutz	08.05.2026	Erstellung basierend auf dem Repository
1.1	Manuel Lutz	12.05.2026	Überarbeitung, Umbenennung und Ergänzung

1. Ziel und Umfang

Dieses Dokument beschreibt den Software Test Plan (STP) für das DPP-API-Projekt. Ziel ist die Planung und Koordination der Systemtests auf Black-Box-Ebene sowie der vorbereiteten Integrationsprüfungen, inklusive Verantwortlichkeiten, Testumgebung, Teststrategie, Testdaten und Abnahmekriterien.

Gedckte Bereiche:

- Backend-API Endpunkte (DIN EN 18222 Mapping)
- Frontend Viewer-Integration
- BaSyx AAS/Registry/Discovery Orchestrierung
- Nicht-funktionale Prüfungen (Fehlerfälle, HTTP-Codes)

Ausgenommen:

- Last-/Performance-Tests (werden separat geplant)
- Sicherheits- und Penetrationstests

2. Referenzen

- System Test Report (STR): [PROJECT/STR/TINF24F_STR_Team_6_0v1.md](#)
- Software Requirement Specification (SRS): [PROJECT/SRS/SRS.md](#)
- API-Spezifikation / Mapping: [PROJECT/SAS/mapping/DPP-Data-Object-Structure.md](#)
- Issue: Creating STP: <https://github.com/DHBW-TINF24F/Team6-BaSyx-DPP-API/issues/92>

3. Testgegenstände

- API-Endpunkte für CRUD, Suche, Mapping, Submodel-Zugriff und Fehlerantworten

- Frontend-Integration für Laden, Navigation, Anzeige und Fehlerszenarien
- BaSyx-Komponenten für Registry-Registration, Discovery und AAS-Transformation
- Testdaten-Skripte und Mock-Harness für reproduzierbare Ausführung

4. Teststrategie

- Teststufen: Unit-Tests, Integrations-Tests und Systemtests; dieser STP spezifiziert vor allem Systemtests und deren Vorbereitung
- Testdesign: Anforderungsbasiertes Testen, Aequivalenzklassenbildung, Grenzwertanalyse, explorative Prüfung und negative Testfälle
- Automatisierung: Mock-basierte Integrationstests (Vitest) als Standard; Real-Backend-Mode via `RUN_REAL_BACKEND_TESTS=true`
- Testdaten: Vorbefüllte Beispiel-DPPs via `SOURCE/frontend/aas-web-ui/scripts/setupTestData.mjs`
- Trennung von Anweisung und Daten: Testschritte sind unabhängig von konkreten Testdaten formuliert
- Fehlerklassifikation: ClientErrors (400), NotFound (404), ServerErrors (5xx)

5. Testumgebung

- Lokale Entwicklung: Node.js (Frontend), Spring Boot (Backend) – Backend auf `http://localhost:8080`
- Mock-Harness: Integrationstest-Mocks im Frontend-Tests-Ordner
- Voraussetzungen: Docker/Compose für optionale Backend-Services; Real-Backend-Skript in `SOURCE/frontend/aas-web-ui/REAL_BACKEND_SETUP.md`
- Erwartete Browserumgebung: aktueller Chromium-basierter Browser für manuelle Sichtprüfung

6. Testaktivitäten

- Erstellung STP
- Review und Freigabe: nach Abstimmung mit SRS und STR sowie den reellen Tests
- Implementierung/Erweiterung Integrationstests: Bereits vollständig erstellt jedoch nicht aktiv da das Backend noch fehlt
- Ausführung Mock-Integrationstests: laufend bei Pull Requests und vor Abgabe
- Real-Backend Testlauf: nach Backend-Fertigstellung und vor finaler Systemtestabnahme

7. Verantwortlichkeiten

- Test Manager / Autor STP: Manuel Lutz (inf24224@lehre.dhbw-stuttgart.de)
- Implementierung Backend Tests: Manuel Lutz (inf24224@lehre.dhbw-stuttgart.de) / Backend-Team
- Frontend Integrationstests: Manuel Lutz (inf24224@lehre.dhbw-stuttgart.de) / Frontend-Team
- Freigabe der Testbasis: Test Manager gemeinsam mit Produkt- und Systemarchitekt

8. Abnahmekriterien

- Alle beschriebenen Testfälle sind mit Testdaten, erwarteten Ergebnissen und Priorität dokumentiert
- Mindestens 10 ausgearbeitete Systemtestfälle decken Backend, Frontend, BaSyx und Fehlerfälle ab
- Mock-Mode: alle vorbereiteten Integrationstests laufen reproduzierbar erfolgreich
- Real-Backend-Mode: alle aktivierbaren Integrationstests laufen ohne Mocking gegen das konfigurierte Backend

- Kritische Fehler (Blocking) müssen behoben sein; nicht-blockierende Auffälligkeiten werden dokumentiert

9. Ausgearbeitete Testfälle

Die folgenden Testfälle sind so formuliert, dass Testanweisung und Testdaten getrennt sind. Sie dienen als systemnahe Black-Box-Prüfung und decken die wesentlichen Äquivalenzklassen sowie negative Fälle ab.

Test-ID	Titel	Testdaten	Schritte	Erwartetes Ergebnis
TC-STP-API-01	Create DPP	Gültiger DPP-Payload mit info und submodels	POST /dpps absenden	201 Created, Rückgabe enthält dppID
TC-STP-API-02	Create DPP mit unvollständigem Payload	Payload ohne verpflichtende Felder	POST /dpps absenden	400 Bad Request mit strukturiertem Fehlerobjekt
TC-STP-API-03	Read DPP by ID	Vorhandene dppId	GET /dpps/{dppId} absenden	200 OK und vollständige DPP-Struktur
TC-STP-API-04	Read DPP by Product ID	Gültige productId aus Testdaten	GET /dppsByProductId/{productId} absenden	Zugeordnetes DPP wird gefunden und angezeigt
TC-STP-API-05	Versionierte Suche	Gültige productId plus ISO-8601-Datum	GET /dppsByProductIdAndDate/{productId}?date=...	Historische oder aktuelle DPP-Version wird korrekt geliefert
TC-STP-API-06	Delete DPP by ID	Vorhandene dppId	DELETE /dpps/{dppId} absenden	204 oder definierter Success-Code, anschliessend nicht mehr auffindbar
TC-STP-FE-01	Viewer lädt DPP	Gültige AAS-/DPP-Referenz	DPP im Frontend aufrufen	Viewer zeigt info und submodels lesbar an
TC-STP-	Navigation im Viewer	DPP mit mehreren	Zwischen Submodellen wechseln	Inhalt wechselt korrekt, aktive

Test-ID	Titel	Testdaten	Schritte	Erwartetes Ergebnis
FE-02		Submodellen		Sektion wird markiert
TC-STP-FE-03	Anzeige fehlender Daten	DPP mit fehlenden optionalen Feldern	Anzeige prüfen	Fehlende Felder werden als N/A oder gleichwertig dargestellt
TC-STP-BX-01	Registry Registration	Gültiges DPP für Registry-Eintrag	POST /registerDPP ausführen	Registry-Eintrag wird erstellt und ist auffindbar
TC-STP-BX-02	Discovery / Routing	Konfigurierter BaSyx-Stack	DPP über Discovery abrufen	Anfrage wird korrekt an BaSyx-Komponenten weitergeleitet
TC-STP-ERR-01	Ungültige Parameter	Falsche dppId oder falscher Query-Parameter	Fehlerrequest senden	404 oder 400 mit strukturierter Fehlerantwort
TC-STP-NFR-01	OpenAPI-Konformität	Aktuelle API-Spezifikation	Response- und Pfadkonformität prüfen	Endpunkte und Statuscodes entsprechen der Spezifikation
TC-STP-NFR-02	Reproduzierbarkeit	Definierte Testdaten und Setup-Skript	Testlauf wiederholen	Ergebnis ist bei identischer Umgebung reproduzierbar

10. Berichterstattung und Artefakte

- Testergebnisse: Vitest-Reports in [SOURCE/frontend/aas-web-ui/tests/reports](#) (bei Integrationstests)
- Testdaten-Skripts: [SOURCE/frontend/aas-web-ui/scripts/setupTestData.mjs](#)
- STR wird nach Testausführung aktualisiert: [PROJECT/STR/TINF24F_STR_Team_6_0v1.md](#)